

EXPERINMENT 1

Implement depth first search algorithm and Breadth First Search algorithm, Use an undirected graph and develop a recursive algorithm for searching all the vertices of a graph or tree data structure.

1. Aim

To implement and analyze the **Depth First Search (DFS)** and **Breadth First Search (BFS)** algorithms using a recursive approach to traverse all vertices of an undirected graph.

2. Theory and Definitions

- **Undirected Graph:** A set of vertices connected by edges where the edges have no direction (bidirectional).
- **DFS (Depth First Search):** A traversal that starts at a root node and explores as far as possible along each branch before backtracking. It naturally uses a stack (or the call stack in recursion).
- **BFS (Breadth First Search):** A traversal that explores all neighbor nodes at the present depth before moving to the next level. While typically iterative (using a queue), it can be implemented recursively by passing a queue through recursive calls.

3. Algorithm Design

A. Recursive DFS

1. Mark the current node as **visited**.
2. Print/Process the node.
3. For every unvisited neighbor of the current node, call the DFS function recursively.

B. Recursive BFS

1. Initialize a **Queue** and add the starting node.
2. **Base Case:** If the queue is empty, return.
3. **Recursive Step:**
 - Dequeue a node and print it.
 - For all its unvisited neighbors, mark them as visited and enqueue them.

4. Complexity Analysis

Algorithm	Time Complexity	Space Complexity
DFS	$O(V + E)$	$O(V)$ (Call Stack)
BFS	$O(V + E)$	$O(V)$ (Queue + Call Stack)

- **V** = Number of Vertices, **E** = Number of Edges.

5. Observations & Results

- **Input Graph:** (Describe your test graph, e.g., A—B, B—C, A—C).
- **DFS Output:** (e.g., 0 1 2 3) — Shows a deep path.
- **BFS Output:** (e.g., 0 1 2 4) — Shows level-by-level discovery.

6. Conclusion

The recursive implementations successfully traversed the undirected graph. While DFS is naturally suited for recursion, the recursive BFS demonstrates that any iterative process involving a state (the queue) can be transformed into a recursive one, though it may risk stack overflow on very deep graphs.

EXPERINMENT 2

Implement A star Algorithm for any game search problem.

1. Aim

To implement the A* search algorithm to solve a game-based pathfinding problem by finding the shortest path between a start node and a goal node while avoiding obstacles.

2. Theoretical Background

The A* algorithm uses a cost function $f(n)$ to determine the next node to explore:

$$f(n) = g(n) + h(n)$$

- **$g(n)$** : The actual cost of the path from the start node to node n .
- **$h(n)$** : The **Heuristic**—an estimate of the cost from node n to the goal. For grid games, **Manhattan Distance** or **Euclidean Distance** is typically used.
- **$f(n)$** : The total estimated cost of the cheapest solution through node n .

3. Implementation Details

- **Data Structures:**
 - **Open List (Priority Queue)**: Stores nodes to be explored, sorted by $f(n)$.
 - **Closed List (Set)**: Stores nodes already evaluated.
 - **Adjacency Map**: Representing the game grid.
- **Heuristic Used**: Manhattan Distance (best for 4-directional movement).
 - $h = |x_1 - x_2| + |y_1 - y_2|$

4. A Algorithm Logic (Pseudo-code)*

1. Add the starting node to the **Open List**.
2. While the Open List is not empty:
 - Find the node with the lowest f on the Open List (call it current).
 - If current is the **Goal**, reconstruct the path and stop.
 - Move current from Open to **Closed List**.
 - For each neighbor of current:
 - If neighbor is an obstacle or in Closed List, skip it.
 - If new path to neighbor is shorter OR neighbor is not in Open List:
 - Set f cost of neighbor.
 - Set neighbor's parent to current.
 - Add neighbor to Open List if it isn't there.

5. Observations & Analysis

- **Efficiency:** A* explored fewer nodes than BFS because the heuristic directed the search toward the goal.
- **Optimality:** The algorithm found the mathematically shortest path.
- **Obstacle Handling:** The path successfully "curved" around walls/obstacles defined in the grid.

6. Conclusion

The A* algorithm was successfully implemented for a game grid. It provides an optimal balance between performance (speed) and accuracy (shortest path), making it the preferred choice for real-time game AI.

EXPERINMENT 3

Implement Greedy search algorithm for any of the following application: Dijkstra's Minimal Spanning Tree Algorithm

1. Aim

To implement the Greedy Search strategy using Prim's Algorithm to find the Minimum Spanning Tree of a weighted, undirected graph, ensuring all vertices are connected with the minimum possible total edge weight.

2. Theoretical Background

A **Minimum Spanning Tree (MST)** is a subset of edges in a graph that connects all vertices without any cycles and with the minimum possible total edge weight.

The Greedy Choice:

At each step, the algorithm grows the spanning tree by choosing the cheapest possible edge that connects a vertex in the "growing tree" to a vertex "outside the tree." This local optimal choice leads to a global optimal solution.

3. Algorithm Logic

1. Maintain two sets: one contains vertices already included in the MST, the other contains vertices not yet included.
2. Assign a key value to all vertices (initialize as ∞), except the starting vertex (initialize as 0).
3. While there are vertices not yet in the MST:
 - Pick a vertex u with the minimum key value that is not yet in the MST.
 - Include u in the MST.
 - Update the key values of all adjacent vertices of u . If the weight of edge (u, v) is less than the current key value of v , update v 's key to the weight of (u, v) .

5. Complexity Analysis

- **Time Complexity:** $O(E \log V)$ when using a Priority Queue (Binary Heap), where E is the number of edges and V is the number of vertices.
- **Space Complexity:** $O(V + E)$ to store the graph and the priority queue.

6. Observations & Results

- The algorithm successfully ignores "expensive" edges that would create cycles.
- The greedy nature is evident as the algorithm always expands via the edge with the absolute lowest weight available at the fringe of the explored territory.
- **Output Result:** (Example) For the provided graph, the MST connects nodes with a total cost of 4.

7. Conclusion

Prim's algorithm demonstrates that a Greedy approach is highly effective for MST problems. By making the locally optimal choice at each step, we arrive at a globally optimal tree that connects all nodes at minimum cost.

EXPERINMENT 4

Implement a solution for a Constraint Satisfaction Problem using Branch and Bound and Backtracking for n-queens problem or a graph coloring problem.

1. Aim

To implement a solution for the N -Queens problem where N queens must be placed on an $N \times N$ chessboard such that no two queens threaten each other.

2. Problem Statement (CSP Formulation)

- **Variables (\$X\$):** $Q_1, Q_2, \dots, Q_n$ (one for each queen/row).
- **Domains (\$D\$):** $\{1, 2, \dots, n\}$ (the column index).
- **Constraints (\$C\$):**
 - No two queens in the same **column**.
 - No two queens on the same **diagonal** (both primary and secondary).

3. Theoretical Background

- **Backtracking:** An incremental approach. It places a queen in the first available valid spot in a row and moves to the next row. If it hits a dead end (no valid spots), it "backtracks" to the previous row and moves the queen there.
- **Branch and Bound (Pruning):** Instead of checking the whole board every time, we use lookup arrays (bounds) for columns and diagonals. If a diagonal is already "bound" by a queen, we prune that entire branch of the search tree immediately.

4. Algorithm Logic (Recursive Backtracking)

1. **Start** in the topmost row.
2. **For** every column in the current row:
 - **Check Bounds:** Is the column or either diagonal currently under attack?
 - **If Safe:**

- Place the queen and "bind" the column and diagonals.
- Move to the next row (Recursion).
- If all queens are placed, return **Success**.
- **Backtrack:** If no solution is found, remove the queen and "unbind" the column/diagonals.

6. Complexity Analysis

- **Time Complexity:** $O(N!)$. While the pruning significantly reduces the number of branches explored, the worst-case remains factorial.
- **Space Complexity:** $O(N)$ for the recursion stack and the bounding arrays.

7. Observations & Results

- For $N=8$, the algorithm found **92 total solutions**.
- The use of boolean arrays for "bounding" (columns and diagonals) made the check $O(1)$ instead of $O(N)$, drastically speeding up the execution compared to a naive search.
- **Graph Coloring Comparison:** Similar to N-Queens, Graph Coloring would use backtracking to assign colors to vertices such that no two adjacent vertices share the same color.

8. Conclusion

The N -Queens problem demonstrates how Backtracking combined with Branch and Bound principles can solve complex Constraint Satisfaction Problems. By pruning impossible paths early, we efficiently navigate a search space that would otherwise be too large for a brute-force approach.

EXPERINMENT 5

Develop an elementary chatbot for any suitable customer interaction application.

1. Aim

To design and implement a basic rule-based chatbot capable of handling standard customer queries, providing information, and simulating a natural conversation flow.

2. Theoretical Background

An elementary chatbot operates on the principle of **Pattern Matching**. It scans user input for specific "keywords" or "intents" and maps them to predefined responses.

- **Input Processing:** Converting text to lowercase and removing punctuation.
- **Knowledge Base:** A dictionary or set of rules containing pairs of patterns and responses.
- **Fallback Mechanism:** A default response when the bot does not understand the input (e.g., "I'm sorry, I didn't quite get that.").

3. System Design

The chatbot follows a simple loop:

1. **Receive** user input.
2. **Normalize** the string (lowercase).
3. **Search** for keywords (e.g., "price", "menu", "hours").
4. **Retrieve** the associated response.
5. **Exit** when the user types "bye" or "exit".

5. Observations & Results

- **Direct Hits:** When the user types "What is on your menu?", the bot identifies the keyword menu and responds correctly.
- **Limitations:** The bot lacks "memory" (it doesn't remember previous lines) and cannot handle typos (e.g., "mennu").
- **Efficiency:** The response time is instantaneous because there is no complex processing or API calls involved.

6. Conclusion

The elementary chatbot successfully demonstrates the core concept of automated customer interaction. While limited, it provides a foundation for more advanced AI. To improve it, one could implement **Regular Expressions (Regex)** for better matching or use an NLP library like **NLTK** or **SpaCy** for entity recognition.

EXPERINMENT 6

Implement any one of the following Expert System :

- Hospitals and medical facilities

1. Aim

To design and implement an Expert System that assists medical staff or patients by diagnosing common ailments based on a set of observed symptoms and clinical rules.

2. Components of the Expert System

1. **User Interface:** The mechanism through which the patient/doctor enters symptoms.
2. **Knowledge Base:** A collection of "IF-THEN" rules derived from medical expertise.
3. **Inference Engine:** The "brain" that processes the facts against the rules to reach a conclusion.
4. **Explanation Facility:** Explains why a certain diagnosis was reached.

3. Rule Base (Knowledge Representation)

The system operates on logical rules such as:

- **Rule 1:** IF (fever) AND (cough) AND (sore throat) THEN (Diagnosis: Common Cold).
- **Rule 2:** IF (fever) AND (body ache) AND (chills) THEN (Diagnosis: Influenza).
- **Rule 3:** IF (rash) AND (itching) AND (no fever) THEN (Diagnosis: Allergic Reaction).

5. Observations & Analysis

- **Inference Method:** This system uses **Forward Chaining**, starting from the facts (symptoms) to move toward a goal (diagnosis).
- **Accuracy:** The system's accuracy is strictly dependent on the quality and quantity of the rules in the Knowledge Base.
- **Certainty Factors:** In advanced systems, we would add "probability" (e.g., "There is an 80% chance you have a cold").

6. Conclusion

The implemented Medical Expert System provides a streamlined way to filter minor ailments and provide immediate advice. While it cannot replace a human doctor, it serves as an excellent triage tool in hospital environments to prioritize urgent cases.